

1. Three Screen Tools

Jason Liu describes three computer-facing surfaces in Codex. The in-app Browser is suited to public pages, artifacts, visual review, and annotation. Chrome is suited to signed-in sites, an existing browser profile, cookies, extensions, or multiple tabs. Computer Use is the broadest option for native applications and work that crosses applications.

His routing rule is simple: prefer a plugin when one can finish the task; if a screen is needed, use the narrowest screen tool that can finish it.

2. Why Narrowness Helps

Narrow tools reduce the state the agent must perceive and the number of actions that can go wrong. A typed plugin operation can identify an object directly. Browser can expose the page and element context. Chrome adds authenticated browser state. Computer Use adds the entire desktop, which increases capability but also latency and ambiguity.

Narrowness is therefore not asceticism. It is an information design principle: represent the task using the interface that makes the relevant state most explicit.

3. Browser as Specification

For visual work, Liu describes a useful loop: ask Codex to create a page, open it in the in-app browser, annotate the actual element, and let the comment carry screenshot and element context back into the thread. The page becomes a shared specification.

This improves on prose such as "the button feels off" because the critique is attached to the rendered object. The agent receives what was seen and where. The same principle generalizes: when the artifact itself can carry feedback, bind the instruction to the artifact rather than redescribing it from memory.

4. Chrome

Chrome is appropriate when authentication or browser-local state is essential. Examples include a signed-in CMS, an internal dashboard, or a site whose behavior depends on extensions. The extra access also raises the stakes. Researching and drafting can often proceed freely; sending, purchasing, publishing, submitting, or changing account settings should normally remain explicit boundaries unless the user has clearly authorized them.

5. Computer Use

Computer Use is the last-mile tool when the work lives in a native application or crosses multiple applications. Kundel's Astra examples show a model combining code with specialized software such as BrickLink Studio and Blender. A task can be partly programmatic and partly visual: generate or transform data with code, inspect it in the application that understands the format, then iterate there.

The important lesson is not "always use Computer Use." It is "do not force every task through text and shell when the domain tool contains essential state or inspection capability."

6. A Routing Table

Need | Preferred surface

Structured external data/action | Plugin or connector

Public page or visual artifact | Browser

Signed-in web state | Chrome

Native application | Computer Use

Cross-application workflow | Computer Use, often with narrower tools for substeps

Codebase transformation | Shell/editor/tools, with screen tools only where runtime or visual evidence matters

7. Boundary Case

Suppose a web application requires login, but the task is only to inspect public CSS after authentication is established by a local fixture. Chrome is not automatically necessary. If the local app can reproduce the state, Browser may be narrower and more deterministic. Tool choice follows required state, not the name of the product being tested.

References: Jason Liu, "Three Ways Codex Can Use a Computer"; OpenAI, Codex computer-use and browser documentation; Dominik Kundel, "Five things I learned from using Astra."

Independent Open Course

<https://github.com/dicnunz/codex-astra-course>

CODEX.00 | Codex with GPT-6 Astra

Handout 10: Browser, Chrome, and Computer Use

Fall 2026

License and source

Structural and pedagogical adaptation of MIT OpenCourseWare 24.00 Problems of Philosophy, Fall 2019, Prof. Miriam Schoenfield.

Original course and this adaptation are licensed CC BY-NC-SA 4.0.

This course is independently produced and is not affiliated with or endorsed by MIT or OpenAI.

Linked external readings and referenced product documentation retain their own copyrights and terms.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

<https://ocw.mit.edu/courses/24-00-problems-of-philosophy-fall-2019/>